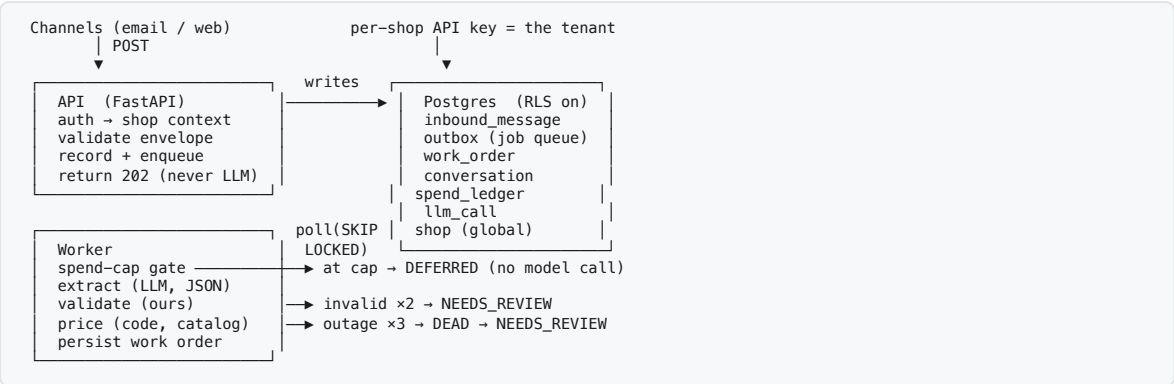
Architecture note — Riverbend Cycles

The shape



The one boundary everything hinges on: the API never calls the LLM. It records the message and returns `202`. All slow/failable work (model calls, pricing, persistence) happens in the worker. A provider outage or a hit spend cap therefore can never block or fail a customer's request — the worst case is always "a human picks it up," never data loss or a wrong work order.

Module boundaries

Module	Owns	May not
`api`	HTTP, auth→shop, envelope validation, enqueue, read	call the LLM, compute money
`core/catalog`	the immutable service catalog	mutate anything
`core/pricing`	**all money**, from `(code, qty)` + catalog	call LLM / DB
`core/schema` + `validator`	the LLM output contract + semantic rules	—
`core/spend`	the monthly cap gate	—
`core/orchestrator`	the processing state machine	talk HTTP
`llm/*`	the only code that imports `openai`; prompt + parse	set prices
`db/repo` + `engine`	all tenant data access, RLS-scoped	leak across shops
`worker`	claim jobs, retry/backoff, park failures	know HTTP

Two separations are load-bearing: **model picks codes, code computes prices** (the model has no price field to set — see below), and **all tenant data goes through `repo` on an RLS-scoped session**.

Data model & tenancy isolation

Every tenant table has a `NOT NULL shop_id` and a Postgres **Row-Level Security** policy: `USING (shop_id = current_setting('app.shop_id'))`, with `FORCE ROW LEVEL SECURITY` so even the table owner is bound. The app connects as a **non-superuser** role, and `tenant_session(shop_id)` sets `app.shop_id` per transaction. Result: isolation is enforced in the database, not in app filters — a query that forgets its `WHERE shop_id` still returns only the current tenant's rows (proved in `tests/test_tenancy.py`). An unset context sees **zero** rows (safe default).

Idempotency is also in the schema: `inbound_message`'s PK is `(shop_id, message_id)`, and `work_order` is unique on `(shop_id, source_message_id)` — the same message twice yields exactly one work order.

Failure handling

- **Duplicates** — composite PK + `INSERT … ON CONFLICT DO NOTHING` (detected via `RETURNING`); the API returns `202 duplicate`.
- **Provider slow/down** — one timeout in the provider; `LLMTimeout` / `LLMUnavailable` → worker backs off and retries up to 3×, then `DEAD` + a `NEEDS_REVIEW` work order. Customer saw `202` at submit regardless.
- **Spend cap** — per-shop monthly `spend_ledger`; the worker checks it *before* the model call. At cap → `DEFERRED`, no call, spend stops. Resets monthly (keyed by `YYYY-MM`).
- **Bad model output** — validated against our Pydantic schema + semantic rules; invalid output is retried once then parked, never saved as a work order.

Testing

28 tests, aimed at what's risky, not at coverage for its own sake: pricing arithmetic (exact `Decimal`, per-wheel/multi-bike), tenant isolation (RLS proven), idempotency, schema + injection validation, and the spend-cap/outage paths. The failure tests use the *real* provider with its fault switch (which short-circuits before the network), so they stay deterministic with no key. The 40-sample `make replay` is the end-to-end integration test and the source of `RESULTS.md`.

Deployment path to GCP

IaC sketch in `infra/` (Terraform). Services and why:

Concern	GCP service	Why
API + Worker	**Cloud Run** (2 services)	scale-to-zero, container parity with local
Database	**Cloud SQL (Postgres)**	managed, and RLS is a Postgres feature
Queue	**Cloud Tasks**	built-in retry/backoff, rate-limit (maps to cap), and task-name dedup — replaces the outbox 1:1
Secrets	**Secret Manager**	`OPENAI_API_KEY`, DB creds
Cap reset / reconcile	**Cloud Scheduler**	monthly tick

The `queue` interface is the only swap point: outbox-poll locally → Cloud Tasks push in prod.

Three decisions I'd most want challenged

1. **Trust the `shop_id` tag over message content for routing** (sample m-1009 says "Hi Lakeshore team!" but is tagged Downtown). I treat the tag as the tenant boundary and only *flag* the mismatch (`wrong_shop_suspected`) rather than reroute — rerouting would cross a tenant boundary on the model's say-so.
2. **DB-outbox instead of a real broker locally.** Simpler, survives restart, and keeps the idempotency story visible in SQL — but it's poll-based and I lean on per-shop polling, which is the first thing I'd replace at scale (see NOT_BUILT).
3. **Spend cap checked against already-accumulated spend** (pre-check, not reservation). The call that crosses the cap still runs — overshoot bounded by one call. Simpler than reserving an estimate; acceptable for a soft cost guardrail.